Day 4 Solutions:

1. Ant On The Boundary

We are given a list of moves (nums), where each element represents a step either forward or backwards. The key observation is:
We start at position 0. We keep adding each move to a running total (prefix sum). Every time this running total becomes 0, it means we have returned to the boundary (origin). So instead of simulating positions separately, we just maintain a cumulative sum and count how many times it becomes zero.

Code:

```
class Solution(object):
    def returnToBoundaryCount(self, nums):
        position = 0
        boundary_count = 0
        for step in nums:
            position += step
            if position == 0:
                boundary_count += 1
        return boundary_count
```

Complexity:

Time Complexity: O(n)
→ We traverse the array once.

Space Complexity: O(1)
→ Only two variables are used.

2. Average Value of Even Numbers That Are Divisible by Three

This We only care about numbers that are divisible by both 2 and 3, which means they are divisible by 6.
While scanning the array once, we can:

- Keep track of how many valid numbers we find using c
- Accumulate their total using sum

At the end:

- If no valid numbers were found (c == 0), the average should be 0
- Otherwise, return the integer average sum / c

Code:

```
class Solution:
    def averageValue(self, nums: List[int]) -> int:
        c=0
        s=0
        for num in nums:
            if num % 2 == 0 and num%3 == 0:
                s+=num
                c+=1
        return s//c if c!=0 else 0
```

Complexity:

- Time complexity: O(N)
- Space complexity: O(1)

3. Maximum Nesting Depth of the Parentheses

To find the nesting depth of the given VPS string s, we can iterate through each character of the string and keep track of the current nesting depth. Whenever we encounter an opening parenthesis '(', we increment the depth, and whenever we encounter a closing parenthesis ')', we decrement the depth. We update the maximum depth encountered so far accordingly.

1. Initialize variables count and max_num to keep track of the current depth and maximum depth encountered so far, respectively. Set both to 0.
2. Iterate through each character i in the input string s.
3. If i is '(', increment count by 1 and update max_num if count exceeds it.
4. If i is ')', decrement count by 1.
5. Finally, return max_num, which represents the maximum nesting depth.

Code:

```
class Solution:
    def maxDepth(self, s):
        count = 0
        max_num = 0
        for i in s:
            if i == "(":
                count += 1
                if max_num < count:
```

```
            max_num = count
        if i == ")":
            count -= 1
    return(max_num)
```

Complexity:

- Time complexity:
  O(n), where n is the length of the input string s. We traverse the entire string once.
- Space complexity:
  O(1), as we only use a constant amount of extra space for variables count and max_num.